

Algorithmic Analysis of Water Distribution Networks

Analysis and Design of Advanced Algorithms (Gpo 605)
TC2038, Situation Problem 2

Santiago Hernández Senosiain
A01787550

November 2, 2026

Contents

1 Problem definition **1**

2 Datasets **2**

3 Algorithm **3**
3.1 T1: graph traversal . . . 4
3.2 T2: geometric computation and dynamic programming . . . 4
3.3 T3: greedy shortest paths for sectorization . . . 4
3.4 T4: shortest-path trees for freshness . . . 5
3.5 T5: network flow . . . 5
3.6 T6: dynamic programming, greedy, local search and branch and bound . . . 5
3.7 T7: divide and conquer for geometric search . . . 6
3.8 Selection hyper-heuristic for T6 . . . 6

4 Implementation **7**
4.1 Code organization . . . 7
4.2 Verification . . . 8

5 Complexity analysis **8**
5.1 Theoretical complexity . . . 8
5.2 Measurements . . . 9
5.3 Agreement with theory . . . 11

6 Results **12**
6.1 T1: representation . . . 12
6.2 T2: pipe lengths . . . 12
6.3 T3: sectorization . . . 13
6.4 T4: freshness . . . 14
6.5 T5: maximum flow . . . 15
6.6 T6: sampling routes . . . 16
6.7 Hyper-heuristic for T6 . . . 18
6.8 T7: expansion . . . 19

7 Conclusions **21**
7.1 What was learned . . . 21
7.2 How the tasks relate . . . 21
7.3 Limitations . . . 22

References **23**

A Sampling walks **24**

1 Problem definition

A water distribution network is an undirected graph $G = (V, E)$ whose vertices are junctions with coordinates $p_v \in \mathbb{R}^2$, a subset $S \subseteq V$ of which are water sources, and whose edges are pipes with a capacity $c(u, v) > 0$. The length of a pipe is $\ell(u, v) = \|p_u - p_v\|_2$, and $d_G(u, v)$ denotes the shortest-path distance through the network under ℓ. The Water Graph Problem (Narváez Terán, 2026) poses seven questions on this model. This report covers all seven as a single study, because their results feed each other: lengths define distances, distances define sectors, and sectors define the flow problems. The seven computational problems, with their input and expected output, are:

T1 Representation. Build an adjacency list and an adjacency matrix of G, explore G with BFS and DFS, find its connected components, and compare the two representations. *Input:* V and E. *Output:* both representations, the traversal orders, the list of components, and the measured time and memory of each representation.

T2 Pipe lengths. Compute every pipe length and compare straight-line with network distance. *Input:* the coordinates and the pipes. *Output:* $\ell(u, v)$ for every pipe, the shortest, longest and total length, the ratio $d_G(u, v)/\|p_u - p_v\|$ for every pair of nodes, and a structure that answers length queries efficiently.

T3 Sectorization. Assign every node to its closest source, $\sigma(v) = \arg\min_{s \in S} d_G(s, v)$, and find the pipes to close, $C = \{(u, v) \in E : \sigma(u) \neq \sigma(v)\}$. *Input:* G, ℓ and S. *Output:* the nodes of each sector $V_s = \{v : \sigma(v) = s\}$, the set C, and a colour-coded map.

T4 Freshness. Water age is taken as proportional to d_G; find the node of each sector that receives the oldest water, $t_s = \arg\max_{v \in V_s} d_G(s, v)$. *Input:* the sectors and distances of T3. *Output:* t_s, $d_G(s, t_s)$ and the route from s to t_s for every sector, and the comparison of distances before and after sectorization.

T5 Maximum flow. Find the largest flow from s to t_s inside each sector such that $|f(u, v)| \leq c(u, v)$ on every pipe and flow is conserved at every other node. *Input:* the open pipes of each sector, their capacities, s and t_s (T4). *Output:* the maximum flow value, the flow on each pipe and the bottleneck pipes (a minimum cut).

T6 Sampling route. Find a minimum-length closed walk on G that starts and ends at the node labelled office and visits every node; nodes and pipes may be repeated, since G need not contain a Hamiltonian cycle. *Input:* G, ℓ and the office node. *Output:* a closed walk over real pipes, its length, and whether it is proven optimal.

T7 Expansion. Link a new point q by one pipe to its nearest non-source node, $\arg\min_{v \in V \setminus S} \|p_v - q\|$, and update the representations. *Input:* G and the coor-

dinates and capacity of each new point. *Output:* the selected node and the length of the new pipe, the updated network, and the cost of the search and of the update.

2 Datasets

The four instances provided with the challenge (NYT, HAN, FOS and PES) are text files with a `V E` header followed by four sections: `[NODES]` with `id x y source_flag`, `[EDGES]` with `u v capacity`, `[OFFICE]` with the id of the office node used in T6, and `[NEW]` with `x y capacity` for each node to insert in T7. Node attributes are therefore the id, the coordinates and the source flag. The only edge attribute is the third column, which the challenge slides call the pipe capacity and label as the diameter in their figures (Narváez Terán, 2026); it is used as the capacity in T5 and never as a length. Lengths are computed from the coordinates in T2. Table 1 summarises the instances after validation.

Table 1: Instances after validation. Src.: sources; Non-src.: other nodes; Comp.: connected components; Loops: independent cycles, $E - V + 1$; Leaves: nodes of degree 1; Max deg.: largest node degree; Caps.: number of distinct capacity values; Office: id of the office node.

Inst.	$\|V\|$	$\|E\|$	Src.	Non-src.	Comp.	Loops	Leaves	Max deg.	Caps.	Office
NYT	20	21	2	18	1	2	2	4	9	5
HAN	32	34	2	30	1	3	3	4	2	29
FOS	37	58	2	35	1	22	1	4	22	4
PES	71	98	2	69	1	28	5	5	14	51

The loader checks every record and logs problems instead of failing. It drops malformed or non-finite values, pipes to unknown nodes, self-loops and non-positive capacities, keeps a repeated pipe only once, and flags coincident coordinates, gaps in the ids, header counts that do not match the data, a missing office, and `[NEW]` points outside the network's bounding box enlarged by 50% on each side. No value is missing or invalid in any file, but three things are unexpected. PES lists pipe 28-30 twice, once as `28 30` and once as `30 28` with the same capacity; it is kept once, so PES has 98 pipes rather than the declared 99, and the capacity of that pipe is not doubled in T5. The PES ids run from 1 to 89 with 18 gaps, which an id-to-index map handles. NYT's second `[NEW]` point, $(6478, 3050)$, lies 4.6 network widths east of NYT's eastern edge and is identical to HAN's second entry, so it is treated as a copy error and not inserted.

All four instances are connected and have exactly two sources. NYT and HAN are almost trees (2 and 3 independent loops), so most nodes have a single route to a source, while FOS and PES are meshed (22 and 28 loops). Six of the eight sources are leaves (both sources of NYT and HAN, source 37 of FOS and source 7 of PES), so a single

pipe joins each of them to the rest of the network. All networks are very sparse: the density $2E/(V(V-1))$ ranges from 0.039 (PES) to 0.111 (NYT). Figure 1 draws the four networks on their original coordinates.

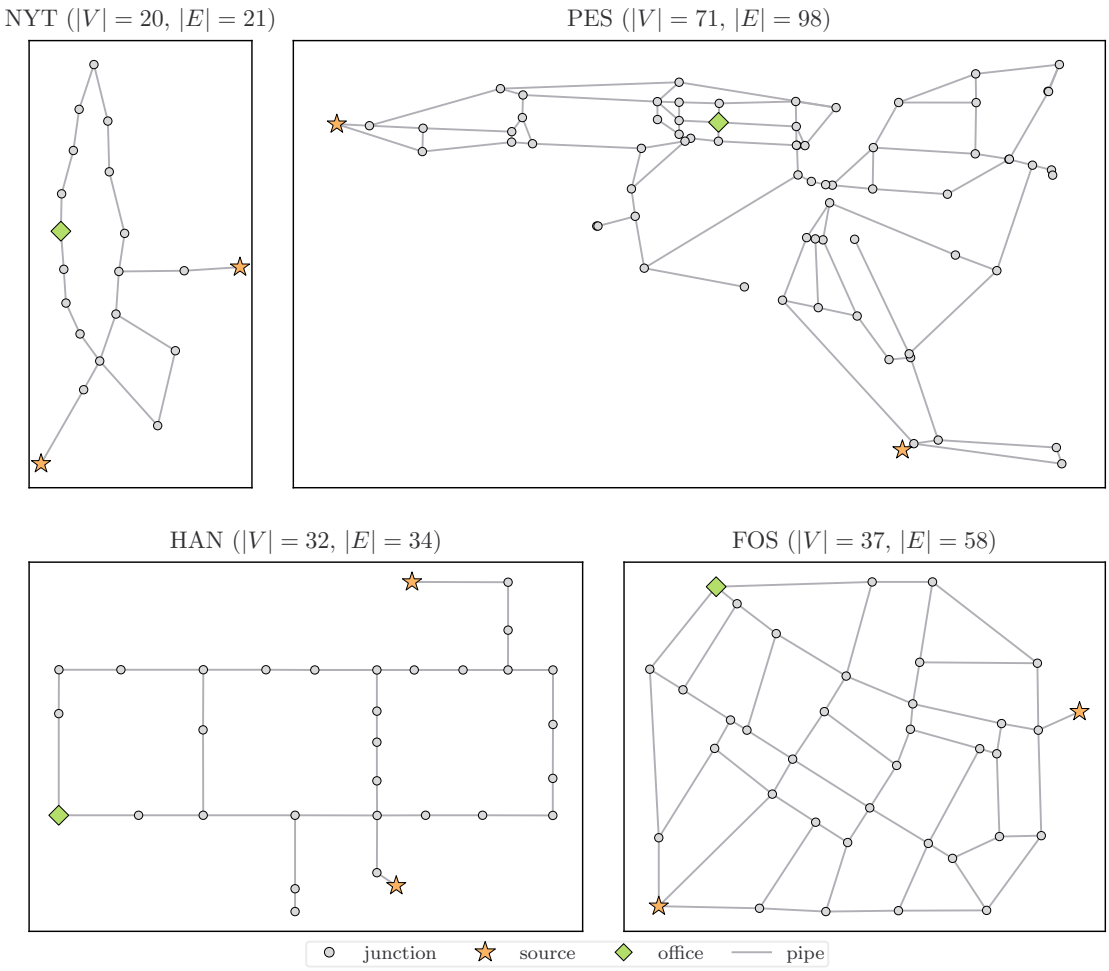


Figure 1: The four instances on their original coordinates, with sources and the office node marked.

Computationally, each instance is held in a `Network` object: a dictionary from node id to coordinates, the list of sources, the list of pipes as (u, v, c) triples, the office and the accepted new nodes. The algorithms run on structures derived from it, an adjacency list and an adjacency matrix (Section 4); T1 compares the two.

3 Algorithm

Each task is matched to a TC2038 technique that fits its structure, and each main algorithm is compared with an alternative or a baseline.

3.1 T1: graph traversal

Connectivity is a reachability question that involves no weights, so graph traversal is sufficient. BFS (with a FIFO queue) and DFS (with an explicit stack) each visit every vertex reachable from the start exactly once; restarting from each vertex not yet visited yields the connected components. Both traversals run on the adjacency list, where the neighbours of a vertex are read directly, and on the adjacency matrix, where the whole row of the vertex must be scanned to find them. The comparison between the two representations is the subject of the task.

3.2 T2: geometric computation and dynamic programming

Each pipe length is one Euclidean distance between its endpoints. Network distances between all pairs of nodes come from Floyd-Warshall, a dynamic program over the set of vertices allowed as intermediate points,

$$D_{ij}^{(k)} = \min\Big(D_{ij}^{(k-1)},\ D_{ik}^{(k-1)} + D_{kj}^{(k-1)}\Big), \tag{3.1}$$

which also records a next-hop matrix from which any route can be reconstructed. All $V(V-1)/2$ pairs are needed and $V \leq 71$, so its $O(V^3)$ cost is small, and the same table later serves as the metric closure of T6. The circuity $d_G(u,v)/\|p_u - p_v\| \geq 1$ measures how much longer the route through the pipes is than the straight line. To query lengths, three structures are compared: a linear scan of the pipe list (baseline), a hash index keyed by the unordered pair of endpoints, and an array of lengths sorted once and searched by bisection, which answers threshold and range queries.

3.3 T3: greedy shortest paths for sectorization

All lengths are non-negative, so Dijkstra's greedy rule (permanently settle the unsettled vertex closest to the source, then relax its pipes) yields exact shortest paths. Multi-source Dijkstra places every source in the priority queue at distance 0. This is equivalent to running Dijkstra from a virtual super-source joined to every source by a zero-length pipe: each node is settled by the source that reaches it first, which is exactly $\sigma(v)$, and the whole assignment costs one Dijkstra run. The alternative runs Dijkstra once per source and assigns each node to the source with the smallest distance. Both variants break exact ties in favour of the source listed first. The set C of pipes whose endpoints lie in different sectors is both necessary and sufficient to separate the sectors, since any open inter-sector pipe would join two of them. Closing C does not lengthen any node's route to its own source: if $\sigma(v) = s$, every node on a shortest path from s to v is also closer to s than to any other source, so the whole path stays inside the sector.

3.4 T4: shortest-path trees for freshness

T4 reuses the multi-source Dijkstra run of T3 instead of recomputing distances. Dijkstra leaves a parent pointer at every node, forming a shortest-path tree rooted at the sources; the farthest node t_s of each sector is an $O(V)$ scan of the distances within V_s, and its route is recovered by following the parent pointers from t_s back to s. To compare distances before and after sectorization, one Dijkstra run per source on the open network gives the farthest node that source would have to reach if it supplied the whole network alone (its eccentricity).

3.5 T5: network flow

Each pipe of a sector becomes two opposite arcs of capacity $c(u, v)$ in a residual graph, so water may flow in either direction. The Ford-Fulkerson method repeatedly finds a path from s to t_s with spare residual capacity and pushes flow along it until no such path remains. By the max-flow min-cut theorem, the nodes still reachable from s in the final residual graph define a minimum cut: the pipes crossing it are saturated, they are the bottleneck, and their capacities add up to the maximum flow. Choosing augmenting paths arbitrarily gives no polynomial bound for real-valued capacities, so two variants with guarantees are compared. Edmonds-Karp always augments along a shortest path in number of arcs, found by BFS. Dinic builds a BFS level graph and saturates it with a blocking flow found by DFS, using current-arc pointers so that no arc is retried within a phase. Shortest path and maximum flow answer different questions: a shortest path minimises the total length of one route and ignores capacity, while a maximum flow ignores length and asks how much water all routes together can carry. A single route carries at most the capacity of its narrowest pipe, whereas the maximum flow is limited by the narrowest cut, which may span several parallel routes.

3.6 T6: dynamic programming, greedy, local search and branch and bound

Let D be the metric closure of the network, $D_{uv} = d_G(u, v)$, computed by Floyd-Warshall. Any feasible sampling walk, read as the order in which it first visits each node, is a tour of D that is no longer than the walk; conversely, any tour of D expands into a walk of the same length by replacing each leg with its shortest path. The optimal sampling walk is therefore the optimal travelling-salesman tour of D starting at the office o, expanded back into real pipes with the next-hop matrix. This problem is NP-hard, so no known exact algorithm is polynomial: an exact method is affordable only for small instances, and heuristics are needed for the rest.

Three methods solve the tour problem. Nearest neighbour (greedy) starts at the

office and always moves to the closest unvisited node. 2-opt (local search) starts from the nearest-neighbour tour and replaces two legs $(a, b), (c, d)$ by $(a, c), (b, d)$ whenever this shortens the tour, until no such exchange exists; the result, called NN + 2-opt below, is deterministic and is a local optimum, with no guarantee. Branch and bound is exact. It searches partial routes from the office depth first, trying the nearest unvisited nodes first, starts with the NN + 2-opt tour as the incumbent, and discards a partial route ending at u with unvisited set R as soon as

$$\mathrm{LB} = \mathrm{cost} + \mathrm{MST}(R) + \min_{r \in R} D_{ur} + \min_{r \in R} D_{ro} \tag{3.2}$$

reaches the incumbent's length. Any completion is a path through all of R, whose length is at least the minimum spanning tree of R, plus one leg from u into R and one leg from R back to o, so the bound never overestimates and pruning never discards the optimum. The search is stopped after 60 s; if it finishes, the incumbent is proven optimal, and otherwise the bound at the root gives a lower limit on the optimum. Only branch and bound guarantees optimality.

3.7 T7: divide and conquer for geometric search

Brute force compares the new point q with all n non-source candidates. A 2-d tree divides the candidates recursively at the median of alternating coordinates. A query descends towards q and visits the other side of a split only if the splitting line is closer to q than the best candidate found so far, which prunes most of the tree when the points are well spread. New points are inserted one at a time in file order with the next free id, as node 38 in the slides' FOS example, and each is linked to the nearest non-source node of the network as it stands, so an earlier new node is also a candidate. The third value of each record is taken as the capacity of the new pipe, and its length is the Euclidean distance. Updating the adjacency list adds one key and two entries, while the adjacency matrix cannot grow in place and must be reallocated and copied.

3.8 Selection hyper-heuristic for T6

A hyper-heuristic searches over heuristics rather than over solutions: a high-level rule chooses, at each step, which low-level heuristic (LLH) to apply, and an acceptance criterion decides whether the result replaces the current solution. A selection hyper-heuristic does this over a fixed set of existing heuristics and can learn online which of them are paying off. Here it works on the same tour problem as T6. The four LLHs are random moves that keep the office first: a 2-opt move (reverse a random segment), a swap of two locations, a relocation of one location, and an or-opt move of a block of 2 or 3 consecutive locations, possibly reversed. Two selection rules are compared. Random selection draws

an LLH uniformly at each step; adaptive selection uses probability matching, choosing each LLH with probability proportional to an exponential average (rate 0.05) of whether its recent applications improved the current tour, with a 5% floor so that no LLH is abandoned. Candidates are accepted by late acceptance hill climbing (LAHC): a candidate is kept if it is no longer than the current tour or than the current tour 50 steps earlier, which lets the search leave local optima without a temperature schedule. Each run applies 30,000 LLHs starting from the nearest-neighbour tour. The baselines are the deterministic NN + 2-opt of T6 and each LLH used alone under the same acceptance and budget, named "LAHC 2-opt move", "LAHC swap move" and so on.

4 Implementation

4.1 Code organization

The code is a Python package, `water`, and one Jupyter notebook per part: `00_data` for the datasets, `01` to `07` for the tasks, `08_performance` for the performance analysis and `09_hyper_heuristic` for the hyper-heuristic. Each notebook runs top to bottom on its own, locally or in Google Colab, and writes the figures and tables used in this report. The four instance files are included in the repository's `data` folder and are read by relative path; in Colab, the first cell of each notebook clones the repository so that the data and the package are available without any manual step. All algorithms are written by hand; NetworkX and SciPy are used only to cross-check their results, and Matplotlib only to draw. Claude Code (Anthropic, 2026) assisted with the writing and the code.

The package has one module per concern. `io` parses and validates the instance files into a `Network` object holding the coordinates, sources, pipes, office and new nodes. `graph` builds the adjacency list (a dictionary from each id to a list of (neighbour, length) pairs) and the NumPy adjacency matrix with an id-to-index map, and runs BFS, DFS and component search on both. `paths` implements Dijkstra with a binary heap and lazy deletion (single, repeated and multi-source) and a vectorised Floyd-Warshall with next-hop matrix. `flow` implements the residual graph, Edmonds-Karp, Dinic and the minimum cut. `tsp` implements nearest neighbour, 2-opt, branch and bound, an exhaustive search used for verification, and the expansion of a tour into a walk. `hh` implements the LLHs, the two selection rules and LAHC. `spatial` implements the brute-force search and the 2-d tree.

The task chain lives in one module, `tasks`: `sectorize` (T3) returns distances, parent pointers, the sector of each node and the open and closed pipes; `farthest` (T4) reads the farthest node and route of each sector from them; and `sector_flows` (T5) builds each sector's flow network from the open pipes with the T4 farthest node as the sink. The module also holds the metric closure used by T2 and T6 and the expansion of T7.

Every notebook that needs a result calls these functions, so the sectors, farthest nodes and sinks are identical across notebooks.

4.2 Verification

Every notebook checks its results before using them (Table 2), and all checks pass on all four instances. As an external check, the computed pipe lengths match to 0.01 the values printed on the course's own figures of FOS, NYT and HAN (Narváez Terán, 2026).

Table 2: Correctness checks, run on all four instances.

Task	Check
T1	List and matrix traversals visit the same vertex set; BFS depths and components equal NetworkX, also on graphs split by removing all pipes of a hub node.
T2	Loop and vectorised lengths agree; Floyd-Warshall equals NetworkX all-pairs Dijkstra; every pipe is the shortest path between its endpoints.
T3	Repeated and multi-source Dijkstra give identical sectors and distances, equal to NetworkX; no exact ties occur; after closing C the components are exactly the sectors and every distance is unchanged.
T4	Each route runs from s to t_s over open pipes and has length $d_G(s, t_s)$; NetworkX Dijkstra on each closed sector finds the same maximum distance.
T5	Edmonds-Karp, Dinic and NetworkX give the same value; capacity and conservation hold on every pipe and node; the minimum-cut capacity equals the flow.
T6	Floyd-Warshall equals repeated Dijkstra; branch and bound equals exhaustive search for $n \leq 9$; every walk is closed at the office, covers all nodes, uses real pipes and has the tour's length.
T7	The 2-d tree equals brute force and SciPy's `cKDTree` on 2,000 random queries per instance and equals brute force on every insertion; the updated list and matrix equal ones rebuilt from scratch.
HH	All 840 runs return valid tours whose reported length is correct and never below a proven optimum; the best tour of each instance expands into a valid walk.

5 Complexity analysis

5.1 Theoretical complexity

The input-size parameters are the number of nodes V, the number of pipes E, the number of sources $|S|$ (2 in every instance), the number of locations to visit n in T6 ($n = V$ for the full problem) and the number of candidate connections n in T7. Table 3 gives the time and space cost of each implementation.

Table 3: Theoretical complexity of the implementations. m: number of pipes reported by a range query; h: depth of the 2-d tree; I: number of hyper-heuristic iterations.

Task	Algorithm	Time	Space
T1	BFS, DFS, components on adjacency list	$O(V+E)$	$O(V+E)$
	BFS, DFS, components on adjacency matrix	$O(V^2)$	$O(V^2)$
T2	all pipe lengths	$O(E)$	$O(E)$
	Floyd-Warshall (all pairs)	$O(V^3)$	$O(V^2)$
	length query, linear scan	$O(E)$	$O(E)$
	length query, hash index	$O(1)$ expected	$O(E)$
	sorting the lengths once	$O(E \log E)$	$O(E)$
	range query on sorted lengths	$O(\log E + m)$	$O(E)$
T3	repeated Dijkstra (binary heap)	$O(\lvert S\rvert (V+E) \log V)$	$O(V+E)$
	multi-source Dijkstra	$O((V+E) \log V)$	$O(V+E)$
T4	farthest node and route, given T3	$O(V)$	$O(V)$
T5	Edmonds-Karp	$O(VE^2)$	$O(V+E)$
	Dinic	$O(V^2E)$	$O(V+E)$
T6	metric closure (Floyd-Warshall)	$O(V^3)$	$O(V^2)$
	nearest neighbour	$O(n^2)$	$O(n)$
	one 2-opt pass	$O(n^2)$	$O(n)$
	branch and bound, MST bound per node	$O(n!\, n^2)$ worst case	$O(n^2)$
	hyper-heuristic	$O(I\, n)$	$O(n)$
T7	brute-force nearest node, per query	$O(n)$	$O(n)$
	2-d tree build	$O(n \log^2 n)$	$O(n)$
	2-d tree query	$O(\log n)$ expected	$O(h)$
	2-d tree insert	$O(h)$	$O(1)$
	graph update, adjacency list	$O(1)$ amortised	$O(V+E)$
	graph update, adjacency matrix	$O(V^2)$	$O(V^2)$

5.2 Measurements

Every time is the median of 21 samples, each sample repeating the call until it lasts at least 2 ms so that timer resolution does not dominate; branch and bound uses the median of 3 runs because single runs take up to seconds. All measurements come from one machine (Python 3.13, Apple arm64). The four instances span only $20 \le V \le 71$, too narrow a range to show growth, so growth was measured on subsets of increasing size: induced PES sub-networks grown breadth-first from both sources ($V = 10$ to 71) for T1 to T5, office-nearest subsets for T6 ($k = 4$ to 16 locations) and random subsets of the PES candidates for T7 ($n = 5$ to 69). Figure 2 compares each measured curve with its cost model scaled by a single fitted constant, and Table 4 gives the times on the four full instances. Absolute times include Python interpreter overhead, which dominates at these sizes; the relevant evidence is the growth rate and the ratio between alternatives.

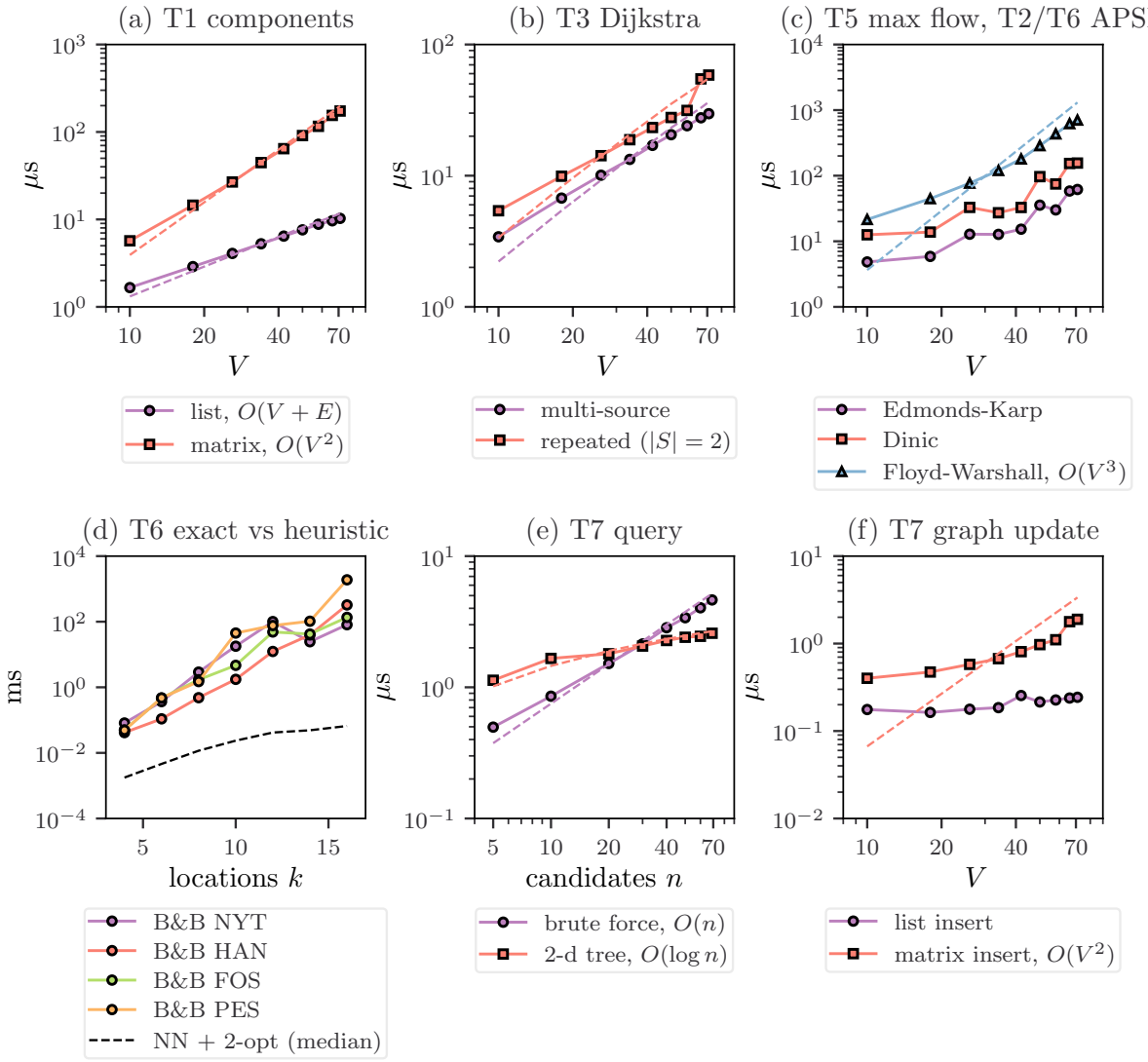


Figure 2: Measured times (solid) against the theoretical cost model scaled to the data (dashed). All axes are logarithmic except the k axis of (d), so that exponential growth in (d) appears as a straight line.

Table 4: Median times in microseconds on the four instances; each column header gives the instance with (V, E). Flow times cover both sectors; NN + 2-opt covers the full route; nearest-node times are for one query.

Algorithm	NYT (20, 21)	HAN (32, 34)	FOS (37, 58)	PES (71, 98)
T1 components, adjacency list	2.8	4.3	5.9	10.4
T1 components, adjacency matrix	16.7	37.2	50.2	173.8
T3 multi-source Dijkstra	7.2	11.1	17.3	29.7
T3 repeated Dijkstra	15.1	24.0	33.5	58.9
T5 Edmonds-Karp, both sectors	12.2	16.9	46.6	60.7
T5 Dinic, both sectors	19.8	30.5	101.0	112.9
T2/T6 Floyd-Warshall	49.9	105.4	138.3	692.8
T6 NN + 2-opt, full route	103.6	268.0	536.0	2,602.0
T7 nearest node, brute force	1.4	2.2	2.4	4.7
T7 nearest node, 2-d tree	1.1	1.4	1.3	2.1

5.3 Agreement with theory

Most measurements follow their models. Component search grows with log-log slope about 0.9 on the adjacency list and 1.8 on the matrix (models 1 and 2), and the matrix is about 17 times slower at $V = 71$, as $O(V^2)$ against $O(V + E)$ predicts for sparse graphs. Both Dijkstra variants follow $(V + E) \log V$, with slopes about 1.1 and 1.2. Repeated Dijkstra performs exactly twice the relaxations of the multi-source version and takes 1.4 times as long on the median subset and 2.0 times on the full PES network, where fixed overheads matter least.

Three measurements deviate from their models, each for an identifiable reason. The Floyd-Warshall time grows with slope about 1.8 instead of 3, because each of its V steps is one vectorised NumPy update whose fixed cost dominates while the matrix has at most 5,041 cells; it still performs exactly V^3 elementary updates, and its curve steepens towards the V^3 reference as V grows. Edmonds-Karp and Dinic grow far more slowly than $O(VE^2)$ and $O(V^2E)$ (Figure 2c), because every source-sink pair needs at most four augmenting paths, each found by one $O(E)$ BFS; at this scale Edmonds-Karp is faster, about 2.5 times on the median sub-network and 1.6 to 2.2 times on the full instances, since Dinic's level graph and recursive blocking-flow search only pay off when a phase contains many augmenting paths. The matrix insertion grows about 5 times from $V = 10$ to 71, much less than V^2, because copying a few thousand cells is cheap next to NumPy's allocation cost; the quadratic growth would dominate on larger networks.

Branch and bound is the only exponential cost. Its time multiplies by about 1.4 (NYT) to 2.2 (HAN) per added location, reaching about 2 s at $k = 16$ on PES, while NN + 2-opt stays below 0.1 ms. The 2-d tree query grows logarithmically (3.5 to 11 distance evaluations) and beats brute force from about 30 candidates, but its build pays for itself only after a few tens of queries, so for the two or three insertions per instance brute force

is sufficient.

6 Results

6.1 T1: representation

Every instance is a single connected component, so every junction can be supplied by at least one source. BFS on the adjacency matrix is about 8 (NYT) to more than 20 (PES) times slower than on the adjacency list, and the gap grows with V because each visited vertex scans a full row of the matrix. Memory is closer than the asymptotic costs suggest at this size: each list entry is a Python tuple holding a neighbour and a float, 115 to 140 bytes per pipe end, against 8 bytes per matrix cell (the matrix totals also include the id-to-index map). The list is therefore slightly larger on NYT (5.7 against 4.7 KiB), but the matrix grows quadratically and is already twice the size of the list on PES (44.6 against 23.3 KiB). The matrix only wins the query "is there a pipe between u and v" (about 67 against 135 ns). Since no task needs that query in an inner loop, the list is the working representation for traversal, Dijkstra and flow, and the matrix is used only where an all-pairs table is the goal (Floyd-Warshall).

6.2 T2: pipe lengths

Table 5 gives the length statistics; the total is the length of pipe each utility maintains. Length and capacity carry different information: their correlation ranges from -0.65 (NYT) to $+0.13$ (HAN), so a long pipe is not necessarily a large one. Circuity separates the instances. The FOS mesh keeps every pair of nodes within 1.75 times its straight-line distance, while the near-trees NYT and HAN reach 4.73 and 3.32, because some nodes that are close on the map are joined only around the tree. PES is the extreme case: nodes 19 and 39 are 147 units apart but 4,146 units apart by pipe, because PES consists of two dense clusters joined by very few pipes. Averaged over all node pairs, the route through the pipes is 19% (FOS) to 83% (PES) longer than the straight line, so straight-line distance would badly underestimate water travel, which is why T3 and T4 use d_G. A pipe is always the shortest route between its own endpoints, since a straight segment cannot be beaten by a polyline. For queries, the hash index answers a length lookup in under 0.1 µs against about 7 µs for a scan of the PES pipe list, and the sorted array finds the 9 PES pipes longer than 1,000 units with 7 comparisons instead of 98.

Table 5: Pipe lengths in coordinate units. Shortest and Longest give the pipe and its length ℓ; Circ.: circuity $d_G(u,v)/\|p_u - p_v\|$, mean and maximum over all node pairs.

Inst.	$\lvert E\rvert$	Total	Shortest	ℓ	Longest	ℓ	Mean	Circ. mean	Circ. max
NYT	21	6,020	9-10	182.2	9-16	480.8	286.7	1.44	4.73
HAN	34	33,827	21-22	399.5	29-30	1780.9	994.9	1.32	3.32
FOS	58	56,154	33-34	240.6	5-6	2284.8	968.2	1.19	1.75
PES	98	42,463	26-17	3.6	9-6	1605.9	433.3	1.83	28.20

6.3 T3: sectorization

Table 6 lists the nodes assigned to each source, and Figure 3 maps them. Each sector is the service area its source supplies when the network is operated as independent districts. The pipes to close are 2-3, 11-9 and 20-16 in NYT; 7-8 and 14-15 in HAN; 18-19 and 19-20 in PES; and eight in the FOS mesh: 3-4, 28-9, 11-19, 27-16, 11-26, 16-29, 26-27 and 28-29. The number of closures reflects how meshed the network is where the two service areas meet: two or three in the tree-like NYT and HAN and in PES, whose clusters meet in a narrow band, against eight in FOS. HAN's sectors are the most unequal (7 against 25 nodes), because source 13 sits behind a single 1,683-unit pipe at the edge of the network, so most nodes are closer to source 1. No node is exactly equidistant from both sources, so the tie-breaking rule never applies. Multi-source Dijkstra performed exactly half the relaxations of the repeated version (196 against 392 on PES) and about half the heap removals.

Table 6: T3 node assignments: the nodes of each sector, identified by its source.

Inst.	Source	Nodes	Node ids in the sector
NYT	17	10	3, 4, 5, 6, 7, 8, 9, 10, 16, 17
NYT	19	10	1, 2, 11, 12, 13, 14, 15, 18, 19, 20
HAN	13	7	8, 9, 10, 11, 12, 13, 14
HAN	1	25	1, 2, 3, 4, 5, 6, 7, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32
FOS	7	19	4, 5, 6, 7, 8, 12, 13, 14, 15, 16, 19, 20, 21, 22, 23, 24, 25, 26, 28
FOS	37	18	1, 2, 3, 9, 10, 11, 17, 18, 27, 29, 30, 31, 32, 33, 34, 35, 36, 37
PES	1	38	1, 2, 3, 4, 5, 10, 11, 19, 31, 32, 37, 38, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 84, 86, 87, 88, 89
PES	7	33	6, 7, 8, 9, 12, 13, 14, 15, 17, 18, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 33, 34, 35, 36, 39, 40, 41, 65, 76, 82, 83, 85

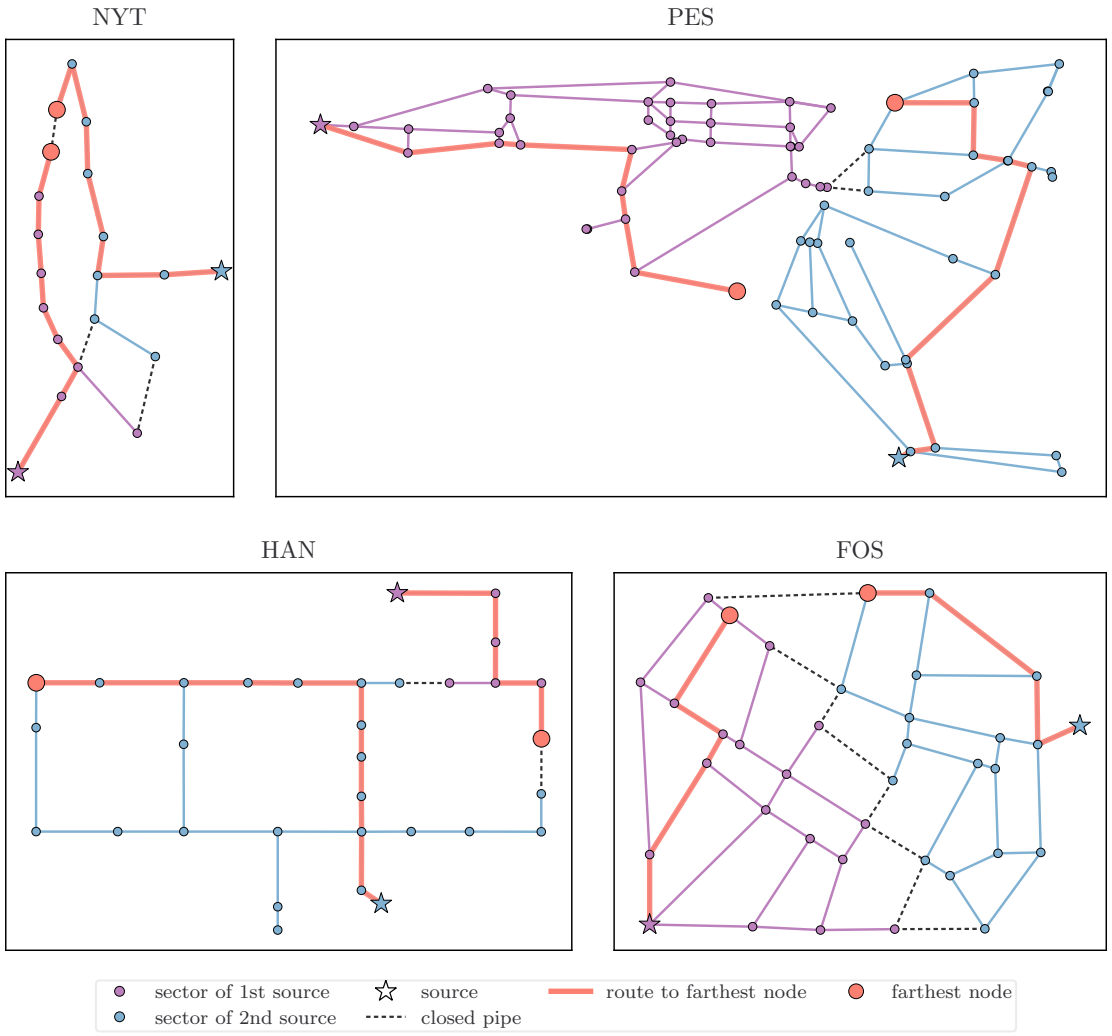


Figure 3: T3 and T4. Sectors from multi-source Dijkstra (node and pipe colour), closed pipes (dashed) and the shortest route from each source to the node of its sector that receives the oldest water.

6.4 T4: freshness

The sectors are those of T3, obtained by multi-source Dijkstra over pipe lengths. Table 8 gives each sector's farthest node and distance, and Table 7 the route to it, also drawn in Figure 3. These nodes receive the oldest water and are the natural places to monitor water quality or to flush. Four of the eight (NYT 3 and 2, HAN 8 and FOS 3) are endpoints of a closed pipe, where the two service areas meet; the other four lie at the far end of their sector (PES 10 is a leaf, HAN 31 the western corner of the network).

Table 7: T4 routes from each source s to its farthest node t_s, as node sequences.

Inst.	s	t_s	Route
NYT	17	3	17-10-9-8-7-6-5-4-3
NYT	19	2	19-18-12-13-14-15-1-2
HAN	13	8	13-12-11-10-9-8
HAN	1	31	1-2-3-19-18-17-16-27-26-25-32-31
FOS	7	12	7-6-21-14-13-12
FOS	37	3	37-1-17-2-3
PES	1	10	1-2-62-58-57-44-42-11-10
PES	7	21	7-6-8-29-35-25-24-17-26-27-23-21

Distances change with sectorization in one sense only. Per node nothing changes: closing C leaves every node's distance to its own source intact (Section 3), so no node receives older water after sectorization. Per source, the comparison is with the farthest node the source would have to reach if it supplied the whole open network alone. Column "Before" of Table 8 gives that distance divided by $d_G(s, t_s)$: sectorization reduces the worst water age each source is responsible for by a factor of 1.49 to 2.25 in FOS and PES and by 2.75 for HAN source 13, which alone would have to reach node 29 at 13,638 units. HAN source 1 is unaffected (1.00), because its farthest node is the same with or without the other source.

Table 8: Results per sector. s: source; $|V_s|$: sector size (T3); t_s and $d_G(s, t_s)$: farthest node and its distance (T4); Before: farthest distance from s in the open network divided by $d_G(s, t_s)$; Max flow: maximum flow from s to t_s inside the sector (T5); Route: capacity of the T4 route alone, that is, of its narrowest pipe; Min cut: the saturated bottleneck pipes, with their capacities.

Inst.	s	$\lvert V_s\rvert$	t_s	$d_G(s, t_s)$	Before	Max flow	Route	Min cut (capacity)
NYT	17	10	3	1,883.3	1.24	73	73	10-17 (73)
NYT	19	10	2	2,101.2	1.11	61	61	18-19 (61)
HAN	13	7	8	4,963.1	2.75	16	16	12-13 (16)
HAN	1	25	31	9,531.2	1.00	16	16	1-2 (16)
FOS	7	19	12	4,931.1	1.49	55	19	5-6 (19), 13-14 (19), 19-20 (17)
FOS	37	18	3	4,131.7	1.82	58.8	17	2-3 (17), 3-11 (41.8)
PES	1	38	10	4,404.1	2.25	151	101	10-11 (151)
PES	7	33	21	4,529.3	2.01	101	101	6-7 (101)

6.5 T5: maximum flow

Table 8 and Figure 4 give the maximum flow from each source to its sector's farthest node. In six of the eight sectors the minimum cut is a single pipe at a leaf: the only pipe leaving the source (NYT, HAN and PES source 7) or entering the sink (PES 10).

There the rest of the sector does not affect throughput, and in five of these six sectors the maximum flow equals what the T4 route carries on its own. Comparing with that route shows concretely that the two problems differ. The shortest route of PES source 1 is limited to 101 by one pipe, but a second route into node 11 lets 151 units reach node 10, saturating the sink's pipe. In the FOS mesh the sector of source 7 is limited by three disjoint pipes that together carry 55, against 19 on the shortest route alone, and the sector of source 37 carries 58.8 against 17.

To see how capacities constrain the flow, the cut pipes were made unlimited and the flow was recomputed; the new value is set by the next bottleneck. The gain is nil in HAN, where every route is a chain of capacity-16 pipes and the next cut is again 16, so upgrading one pipe would achieve nothing. Elsewhere it ranges from 15% (FOS source 7, 55 to 63) to a factor of 3.6 (FOS source 37, 58.8 to 211), which tells an engineer which pipes to enlarge first and how much would be gained. Edmonds-Karp and Dinic always return the same value; no sector needed more than 4 augmenting paths, and Dinic no more than 3 phases.

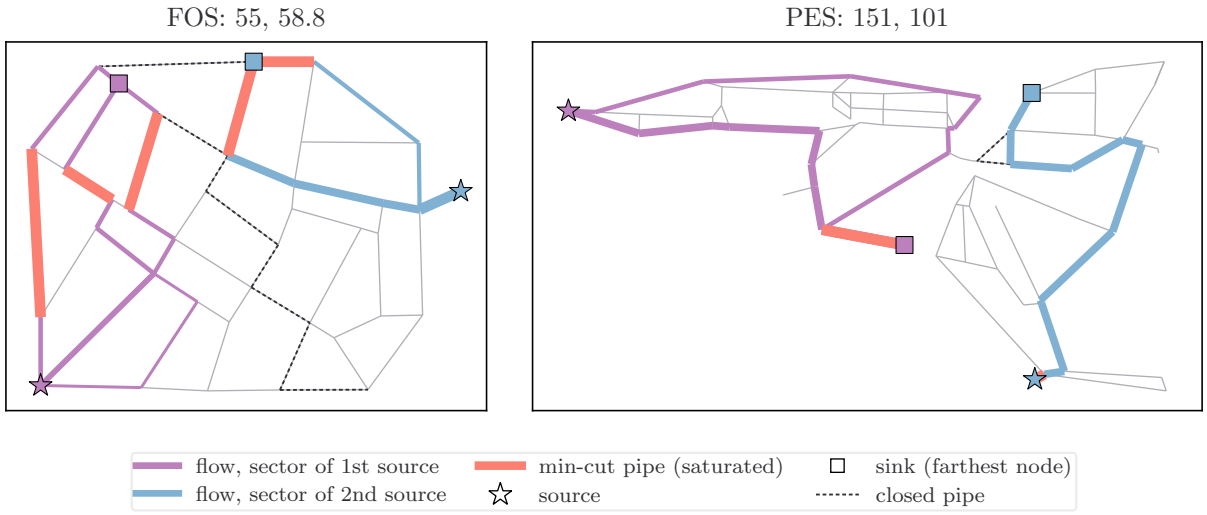


Figure 4: T5 on the two meshed instances. Pipe width is proportional to the flow it carries; saturated minimum-cut pipes are highlighted. Panel titles give the maximum flow of each sector. In NYT and HAN the cut is the single pipe at the source (Table 8).

6.6 T6: sampling routes

Table 9 compares the three methods on the full instances. NYT is solved exactly: the optimal walk from the office (node 5) has length 7,072 and traverses 24 pipes,

5-4-3-2-1-15-14-13-12-18-19-18-12-11-20-16-9-10-17-10-9-8-7-6-5,

going out and back along the two branches that end at the sources (Figure 5a). The walk is longer than the network's total pipe length (6,020) because those dead ends must be walked twice. For HAN, FOS and PES branch and bound does not finish within

60 s, so their routes are NN + 2-opt local optima: approximate and not certified. The hyper-heuristic of Section 6.7 later shortens the PES route to 34,700; the other two are not improved. The expanded walks of all four instances are listed in Appendix A. The root lower bound is too weak to close the gap, because the MST counts each connection once while the walk must come back out of every dead end (HAN needs 43 traversals of its 34 pipes).

Table 9: Sampling routes on the full instances (lengths in coordinate units). NN: nearest neighbour; NN + 2-opt: 2-opt local search from the NN tour; B&B: best tour after branch and bound; Status: optimal if the search finished within 60 s; Lower bound: bound at the root of the search; NN + 2-opt gap: how far the NN + 2-opt route lies above the optimum when the optimum is proven, and otherwise an upper limit ($\leq$) computed against the lower bound, which is loose because the bound is weak; Walk pipes: pipe traversals of the expanded walk.

Inst.	n	NN	NN + 2-opt	B&B	Status	Lower bound	NN + 2-opt gap	Walk pipes
NYT	20	9,581	7,622	7,072	optimal	5,535	7.8%	24
HAN	32	43,802	40,810	40,810	best found	30,605	$\leq$ 33.3%	43
FOS	37	40,130	30,424	30,424	best found	25,147	$\leq$ 21.0%	41
PES	71	38,604	35,510	35,510	best found	22,195	$\leq$ 60.0%	97

To measure the heuristics where the optimum can be computed, the problem was restricted to the k nodes closest to the office (by network distance, office included, walking on the full network) for $k = 4, 6, \ldots$ until a subset needed more than 100,000 search nodes (a machine-independent budget; at most 300,000 per subset); this solved subsets of up to 18 (PES) to 26 (FOS) nodes. On the 40 solved subsets nearest neighbour is on average 16.0% above the optimum (worst 37.8%) and NN + 2-opt 1.9% (worst 15.8%), reaching the optimum in 31 of them (Figure 5c). On the full instances the nearest-neighbour route is 7% (HAN) to 32% (FOS) longer than the NN + 2-opt route, which needs between 1 and 11 improving exchanges.

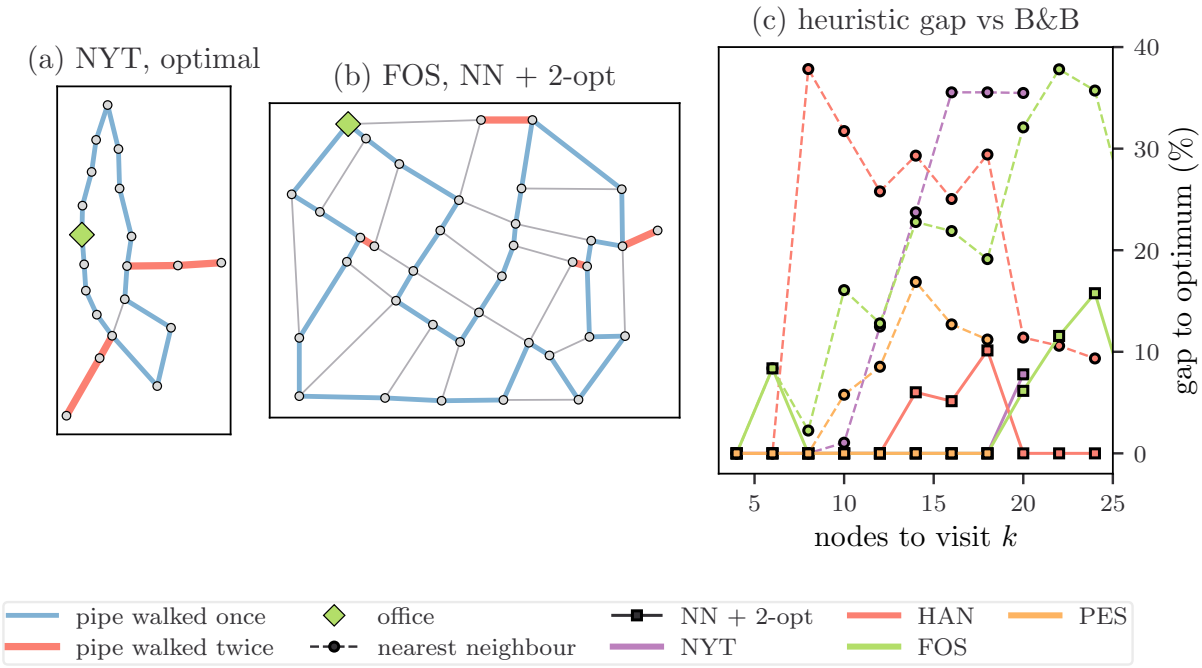


Figure 5: T6. (a) Optimal NYT walk (7,072) and (b) best FOS walk found by NN + 2-opt (30,424), both from the office and expanded into real pipes; pipes walked twice are highlighted. (c) Gap to the branch-and-bound optimum of nearest neighbour (dashed) and NN + 2-opt (solid) on office-nearest subsets of k nodes.

6.7 Hyper-heuristic for T6

The hyper-heuristic of Section 3.8 was run with 10 seeds per configuration on 14 problems: the 11 with a proven optimum (full NYT and office-nearest subsets of 12, 16 and 20 nodes where branch and bound finishes within 300,000 search nodes) and the full HAN, FOS and PES instances (`09_hyper_heuristic`). All 840 runs return valid tours and none is shorter than a proven optimum. On the problems with a proven optimum (Figure 6b) both hyper-heuristics average about 0.2% above it and are optimal in about 94% of runs, slightly ahead of the best single move (LAHC 2-opt move, 0.29%) and with a smaller worst case (4.66% against 7.96%), while the deterministic NN + 2-opt averages 1.73%. The swap and relocate moves alone stay 11 to 14% above the optimum, because they cannot undo crossing legs, which a 2-opt move removes directly.

The main gain is consistency on the full instances. The random-selection hyper-heuristic reaches the best known HAN and FOS routes (40,810 and 30,424) in 9 and 6 of its 10 runs, so its median equals the best, whereas the LAHC 2-opt move alone reaches them in 3 and 2 runs and has medians 2.2% and 5.5% above them. On PES it finds the shortest route of the study, 34,700, 2.3% below the T6 route, although its median over seeds (35,200) is slightly worse than that of the LAHC 2-opt move alone (35,064), so mixing in weaker moves costs some consistency there (Figure 6a). Adaptive selection ends up choosing the four moves almost uniformly (about 25% each) and brings no benefit

over random selection: near a local optimum every move rarely improves the tour, so the credit of each decays to the same floor and the rule cannot tell them apart. A run of 30,000 iterations takes about 0.17 s. The HAN, FOS and PES routes remain uncertified.

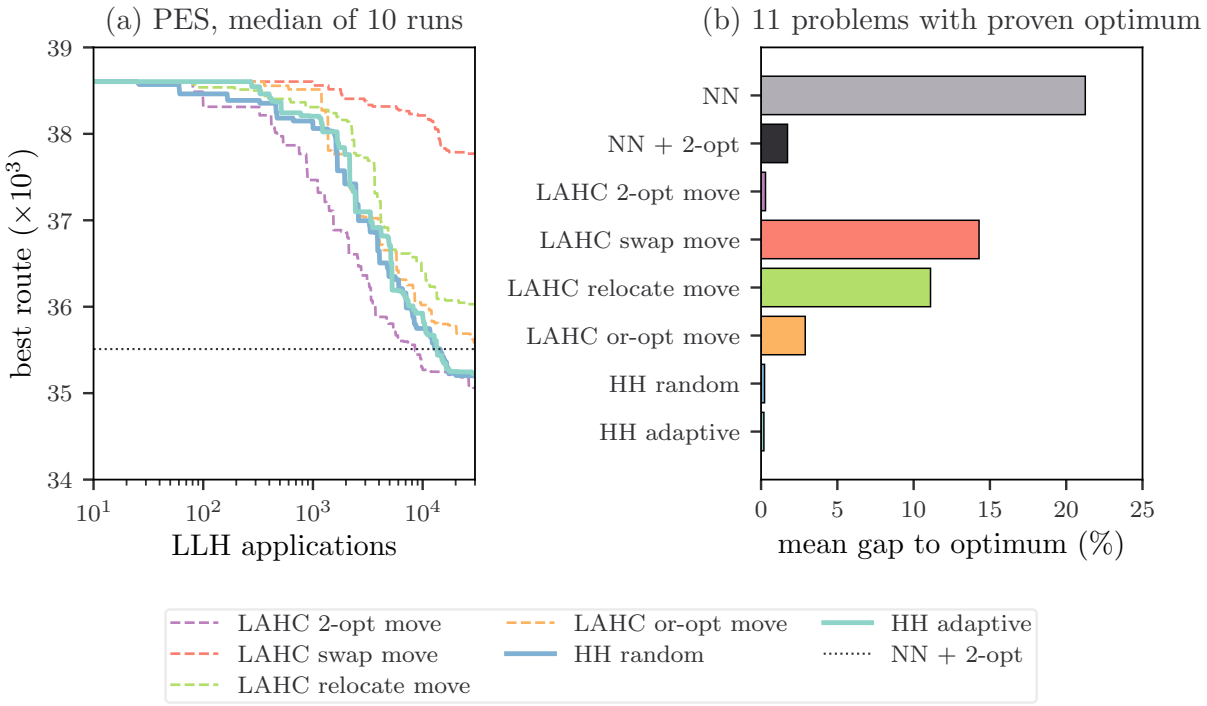


Figure 6: Selection hyper-heuristic for T6. (a) Best PES route over the run (median of 10 seeds) for each move used alone under LAHC (dashed) and for the hyper-heuristics (solid); the dotted line is the deterministic NN + 2-opt route. (b) Mean gap to the proven optimum over the 11 problems where it is known.

6.8 T7: expansion

Table 10 lists the insertions and Figure 7 the expanded networks. The FOS point (7071, 9055) becomes node 38 linked to node 17, the connection drawn in the challenge slides. Each new connection is a single pipe of 150 to 809 units, of the same order as the existing pipes (Table 5), so every new node is served by extending the network at one junction; sources are never used as connection points. No new node lands closest to an earlier new node, so the insertion order does not change any connection. On 2,000 random queries the 2-d tree evaluated on average 6.7 of the 18 candidates on NYT and 11.0 of the 69 on PES, and the fraction falls as the network grows. Each insertion writes one key and two entries into the adjacency list, whereas the matrix copies its whole table (10,225 cells for the two PES insertions).

Table 10: New nodes in file order. Linked to: nearest non-source node; Length: new pipe length; Capacity: third value of the record; Tree eval. and Brute eval.: distance evaluations of the 2-d tree and of brute force.

Inst.	New id	Coordinates	Linked to	Length	Capacity	Tree eval.	Brute eval.
NYT	21	(930, 1779)	13	150.0	120	4	18
NYT	22	(1269, 1069)	20	217.0	170	4	19
HAN	33	(8457, 5177)	9	353.0	18	5	30
HAN	34	(6478, 3050)	5	607.7	20	9	31
HAN	35	(595, 5929)	32	793.5	17	6	32
FOS	38	(7071, 9055)	17	617.1	50	5	35
FOS	39	(1957, 9160)	5	809.1	45	7	36
PES	90	(657029, 963601)	3	363.7	120	9	69
PES	91	(662127, 961448)	25	645.7	105	10	70

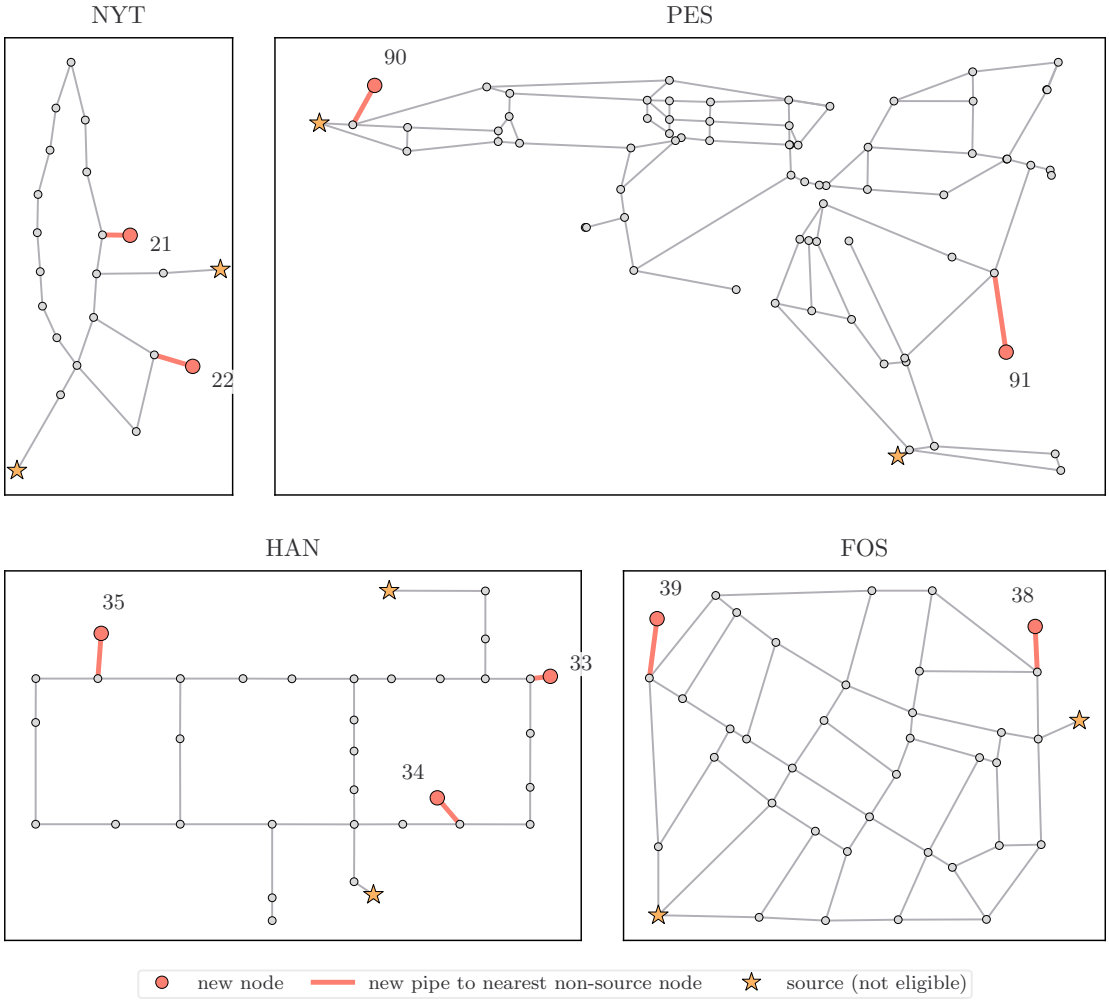


Figure 7: T7. Expanded networks: each new node (labelled with its id) is linked to its nearest non-source node.

7 Conclusions

7.1 What was learned

The seven tasks show that the right technique follows from the structure of each question, not from the data. Connectivity needs only traversal, and on networks this sparse the adjacency list beats the matrix in time and, beyond a few dozen nodes, in memory. Distances need shortest paths, and the greedy choice of Dijkstra is exact because lengths are non-negative; recasting sectorization as a single multi-source run halves its work without changing the answer. Throughput is a different question from distance and needs network flow: on the same source-sink pairs, the shortest route and the maximum flow are set by different pipes, and the minimum cut names the pipes worth upgrading. Nearest-node search is where divide and conquer pays off in principle, but at a few dozen candidates brute force is just as good.

T6 is the only hard problem, and it is where the trade-off between exactness and cost becomes concrete. Branch and bound proves optimality, but its time grows exponentially, so it finished only for NYT and for subsets of at most 26 nodes; for HAN, FOS and PES the routes are heuristic. The cheap heuristics recover most of the value of the exact method: NN + 2-opt was 1.9% above the optimum on average, and the selection hyper-heuristic over random tour moves made the routes more reliable across runs and shortened the best PES route by 2.3%, still without a guarantee. Among the polynomial steps, Floyd-Warshall ($O(V^3)$) is the most expensive; Dijkstra, the flow algorithms and the nearest-node search are all fast at this scale, and the flow algorithms run far below their worst-case bounds because no sector needs more than four augmenting paths.

7.2 How the tasks relate

The tasks form a chain. The lengths of T2 are the weights of every shortest path; T3 turns them into sectors; T4 reads the node with the oldest water in each sector from the same Dijkstra tree; and T5 uses that node as the sink of a flow problem restricted to the sector. T6 reuses the all-pairs distances of T2 as its metric closure, T1 decides the representation all of them run on, and T7 changes the network that all of them analyse. The results also explain one another. The leaf sources found in the data are the dead ends that the T6 walk must traverse twice, and five of the six are also the single-pipe bottlenecks of T5 (FOS source 37 is limited further downstream). The extreme circuity of PES in T2 reflects two clusters joined by few pipes, which is why separating its sectors in T3 takes only two closures. Running T4 and T5 on the same source-sink pairs shows concretely that a shortest route and a maximum flow are different problems: the route minimises length, the flow is set by the narrowest cut, and in the meshed FOS the flow is 2.9 and 3.5 times what the shortest route alone could carry.

7.3 Limitations

The model is purely graph-theoretic: there is no hydraulics (pressure, head loss or demand), capacities are taken as given numbers without units, and water age is taken as proportional to distance rather than to travel time. Flow is analysed for one source-sink pair per sector inside the closed sector, not for the whole network. Several conventions were needed and are stated where they are used: keeping PES's repeated pipe once, rejecting NYT's outlying new point, inserting new nodes in file order, and breaking ties towards the earlier source (no tie occurred). The branch-and-bound bound is simple and weak on near-tree networks, so the full HAN, FOS and PES routes remain uncertified. Finally, the instances are small ($V \leq 71$), so the performance conclusions rest on growth measured across subsets and are affected by Python interpreter overhead.

References

Anthropic (2026). Claude code. AI coding assistant, `https://www.anthropic.com/claude-code`. Used for assistance with writing and code.

Narváez Terán, V. (2026). Problem situation 2: Distribution of water. Lecture slides, Análisis y diseño de algoritmos avanzados, Tecnológico de Monterrey. Includes the NYT, HAN, FOS and PES network instances.

A Sampling walks

Table 11 lists the T6 walks of Table 9, expanded from the tour of the metric closure into consecutive pipes. Each starts and ends at the office and visits every node; repeated nodes are where the walk passes through a junction again.

Table 11: T6 closed walks from the office, as node sequences (optimal for NYT, NN + 2-opt for the others).

Inst.	Length	Closed walk from the office
NYT	7,072	5-4-3-2-1-15-14-13-12-18-19-18-12-11-20-16-9-10-17-10-9-8-7-6-5
HAN	40,810	29-28-23-20-21-22-21-20-3-2-1-2-3-19-18-17-18-19-3-4-5-6-7-8-9-10-11-12-13-12-11-10-14-15-16-27-26-25-24-25-32-31-30-29
FOS	30,424	4-5-13-14-20-14-21-6-7-24-8-28-29-30-9-36-35-34-33-34-31-1-37-1-17-2-3-2-18-10-32-27-16-25-23-22-15-26-11-19-12-4
PES	35,510	51-48-47-56-57-58-62-59-62-2-4-3-1-3-5-61-60-61-55-86-55-54-84-88-89-53-52-45-52-50-46-32-49-32-31-37-38-19-20-21-23-22-85-82-83-26-27-26-17-24-25-12-39-34-41-40-9-33-30-28-29-8-76-65-6-7-6-8-29-35-36-35-25-24-14-15-14-24-17-13-18-19-38-37-31-11-10-11-42-43-87-43-42-44-56-47-48-51